

Video model

May 26, 2026

```
[2]: import os
import torch
import numpy as np
from tqdm.auto import tqdm
from torch import nn
from torch.optim import AdamW
from torch.utils.data import DataLoader
from torchvision import datasets
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

from transformers import AutoImageProcessor, SwinForImageClassification
```

```
[ ]: import kagglehub
path = kagglehub.dataset_download("saitejareddy050/video-dataset")
```

```
[ ]: from IPython.display import FileLink
FileLink(r'/kaggle/working/swin_best_temporal.pth')
```

```
[ ]: import kagglehub
path = kagglehub.model_download('saitejareddy050/swin-models/pyTorch/default/1')
```

```
[ ]: import kagglehub
path = kagglehub.dataset_download("saitejareddy050/video-dataset")
```

```
[3]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

```
[4]: class SwinTemporalTransformer(nn.Module):
    def __init__(self, input_dim=1792): # 1024 (Swin) + 768 (Freq)
        super().__init__()
        # Positional encoding for sequence order
        self.pos_emb = nn.Parameter(torch.randn(1, 16, input_dim) * 0.02)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=input_dim, nhead=8, batch_first=True
        )
```

```

self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=4)

self.classifier = nn.Sequential(
    nn.LayerNorm(input_dim),
    nn.Linear(input_dim, 2)
)

def forward(self, x):
    # x shape: [Batch, 16, 1792]
    x = x + self.pos_emb
    x = self.transformer(x)
    # Max pool across the time dimension [cite: 757]
    x, _ = torch.max(x, dim=1)
    return self.classifier(x)

```

```

[5]: import torch.nn as nn

class SwinTemporalTransformer(nn.Module):
    def __init__(self, input_dim=1792, num_heads=8, num_layers=4):
        super(SwinTemporalTransformer, self).__init__()
        # 1. Temporal Positional Encoding (for 16 frames)
        self.pos_embedding = nn.Parameter(torch.randn(1, 16, input_dim) * 0.02)

        # 2. Transformer Encoder Layers
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=input_dim,
            nhead=num_heads,
            dim_feedforward=2048,
            activation='gelu',
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer,
        ↪ num_layers=num_layers)

        # 3. Final Video Classifier (Fusion Head)
        self.classifier = nn.Sequential(
            nn.LayerNorm(input_dim),
            nn.Linear(input_dim, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, 2)
        )

    def forward(self, x):
        # x shape: [Batch, Frames, 1792]
        x = x + self.pos_embedding
        x = self.transformer(x)

```

```

    # Use Max Pooling to catch transient glitches (flickers)
    x, _ = torch.max(x, dim=1) # [Batch, 1792]
    return self.classifier(x)

```

```

[6]: import torch
import torch.nn as nn
import torch.fft

class FrequencyBranch(nn.Module):
    def __init__(self, in_channels=3, embed_dim=768):
        super(FrequencyBranch, self).__init__()
        self.freq_extractor = nn.Sequential(
            # Stage 1: 64 filters
            nn.Conv2d(in_channels, 64, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),

            # Stage 2: 128 filters
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),

            # Stage 3: 768 filters (Matches your checkpoint's DNA)
            nn.Conv2d(128, 768, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(768),
            nn.ReLU(inplace=True),

            nn.AdaptiveAvgPool2d((1, 1))
        )

    def forward(self, x):
        # 2D FFT and Log Scaling
        fft_complex = torch.fft.fft2(x, norm="ortho")
        fft_shifted = torch.fft.fftshift(fft_complex, dim=(-2, -1))
        magnitude = torch.abs(fft_shifted) + 1e-8
        log_magnitude = torch.log(magnitude)

        # Feature Extraction
        freq_features = self.freq_extractor(log_magnitude)
        return torch.flatten(freq_features, 1) # Returns [Batch, 768]

class SwinFusionClassifier(nn.Module):
    def __init__(self, spatial_dim=1024, freq_dim=768, num_classes=2):
        super().__init__()
        # Total Input: 1024 (Swin) + 768 (Freq) = 1792
        self.classifier = nn.Sequential(

```

```

        nn.Linear(spatial_dim + freq_dim, 512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        nn.Linear(512, num_classes)
    )

    def forward(self, spatial_features, freq_features):
        # Concatenate on the feature dimension
        fused = torch.cat((spatial_features, freq_features), dim=1)
        return self.classifier(fused)

```

```

[7]: import torchvision.transforms as transforms

# Define the transformations required for the Swin Transformer
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

```

[19]: import os
import cv2
import torch
from torch.utils.data import Dataset
from PIL import Image

class OmniVideoDataset(Dataset):
    """
    Dataset class that reads raw video files (.mp4, .avi) on the fly,
    uniformly samples exactly `num_frames` from each video, and prepares
    the sequential frame tensors for the Omni Spatio-Temporal Model.
    """
    def __init__(self, video_paths, labels, num_frames=16, transform=None):
        """
        Args:
            video_paths (list): List of absolute string paths to the video
            ↪ files.
            labels (list): Integer class labels (0 for Fake, 1 for Real).
            num_frames (int): Number of temporal frames required by the
            ↪ Transformer.
            transform (callable): Image transforms applied to individual frames.
        """
        self.video_paths = video_paths
        self.labels = labels

```

```

self.num_frames = num_frames
self.transform = transform

def __len__(self):
    return len(self.video_paths)

def __getitem__(self, idx):
    video_path = self.video_paths[idx]
    label = self.labels[idx]

    # 1. Open video stream using OpenCV
    cap = cv2.VideoCapture(video_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    frames_tensors = []

    # 2. Defensive Fallback: If video file is unreadable/corrupt
    if total_frames <= 0 or not cap.isOpened():
        print(f" [DATA LAYER ALERT] Cannot read video track: {video_path}.␣
↪Creating blank canvas fallback sequence.")
        cap.release()
        frames_tensors = [torch.zeros(3, 224, 224) for _ in range(self.
↪num_frames)]
        return torch.stack(frames_tensors), label

    # 3. Compute uniform temporal sample indexes across the video space
    # Spread exactly 16 frame indices from index 0 to (total_frames - 1)
    sampled_indices = np.linspace(0, total_frames - 1, self.num_frames,␣
↪dtype=int)

    # 4. Extract frames sequentially based on calculated indices
    current_frame_idx = 0
    success = True

    for target_idx in sampled_indices:
        # Fast forward to target frame index context
        while current_frame_idx < target_idx:
            success = cap.grab() # Grabs the frame without decoding it␣
↪(highly optimized performance)
            current_frame_idx += 1
            if not success:
                break

        # Decode and retrieve the target frame matrix
        success, frame_bgr = cap.retrieve()
        current_frame_idx += 1

```

```

        if success and frame_bgr is not None:
            # Convert BGR OpenCV layout to RGB Pillow Format
            frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
            img = Image.fromarray(frame_rgb)

            if self.transform:
                img = self.transform(img)
                frames_tensors.append(img)
            else:
                # Handle unexpected early stream termination gracefully using
                ↪ the last decoded frame
                if len(frames_tensors) > 0:
                    frames_tensors.append(frames_tensors[-1])
                else:
                    frames_tensors.append(torch.zeros(3, 224, 224))

        cap.release()

        # 5. Ensure that we always return exactly `num_frames` elements
        while len(frames_tensors) < self.num_frames:
            frames_tensors.append(frames_tensors[-1] if len(frames_tensors) > 0
            ↪ else torch.zeros(3, 224, 224))
            frames_tensors = frames_tensors[:self.num_frames]

        # Stack array frames sequentially across temporal dimension
        video_sequence = torch.stack(frames_tensors) # Output Shape: (16, 3,
        ↪ 224, 224)
        return video_sequence, label

```

```

[20]: import os

# Define the correct dataset_base_path (from earlier execution)
dataset_base_path = ''

# Set the correct dataset_root based on discovery
dataset_root = os.path.join(dataset_base_path, 'Unified_Video_Dataset')
print(f"Using dataset root: {dataset_root}")

train_dataset = OmniVideoDataset(os.path.join(dataset_root, "Train"),
    ↪ num_frames=16, transform=transform)
val_dataset = OmniVideoDataset(os.path.join(dataset_root, "Validation"),
    ↪ num_frames=16, transform=transform)
test_dataset = OmniVideoDataset(os.path.join(dataset_root, "Test"),
    ↪ num_frames=16, transform=transform)

# Maintain your 40% subsetting logic if desired
subset_size = int(1 * len(train_dataset))

```

```

train_subset, _ = torch.utils.data.random_split(
    train_dataset, [subset_size, len(train_dataset) - subset_size]
)

val_subset_size = int(1 * len(val_dataset))
validation_subset, _ = torch.utils.data.random_split(
    val_dataset, [val_subset_size, len(val_dataset) - val_subset_size]
)

```

Using dataset root: Unified_Video_Dataset

```

-----
TypeError                                Traceback (most recent call last)
Cell In[20], line 10
      7 dataset_root = os.path.join(dataset_base_path, 'Unified_Video_Dataset')
      8 print(f"Using dataset root: {dataset_root}")
----> 10 train_dataset = OmniVideoDataset(os.path.join(dataset_root,
    ↪ OmniVideoDataset(os.path.join(dataset_root, ), num_frames=16, transform=transform)
      11 val_dataset = OmniVideoDataset(os.path.join(dataset_root, "Validation"),
    ↪ num_frames=16, transform=transform)
      12 test_dataset = OmniVideoDataset(os.path.join(dataset_root, "Test"),
    ↪ num_frames=16, transform=transform)

TypeError: OmniVideoDataset.__init__() missing 1 required positional argument:
    ↪ 'labels'

```

```

[10]: batch_size = 2

train_loader = DataLoader(
    train_subset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=4,      # VERY IMPORTANT
    pin_memory=True,    # IMPORTANT
    persistent_workers=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=4,
    pin_memory=True,
    persistent_workers=True
)

```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=4,
    pin_memory=True,
    persistent_workers=True
)
```

```
[11]: from torch import amp
      scaler = amp.GradScaler()
```

```
[13]: import os
      import torch
      import torch.nn as nn
      import kagglehub
      from transformers import SwinForImageClassification

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# =====
# 1. DOWNLOAD FROM KAGGLEHUB (Colab Native)
# =====
# This downloads your exact container from Kaggle
# model_handle = "saitejareddy050/swin-models/pytorch/default"
# print("Fetching model directory from Kaggle...")
# model_checkpoint_dir = kagglehub.model_download(model_handle)

# If your files are nested inside a version folder (like "/1/"), kagglehub
↳ handles it.
# Let's map the precise file names from your logs:
swin_checkpoint_path = "checkpoints/swin_best_model.pth"
freq_branch_checkpoint_path = "checkpoints/freq_branch_standalone.pth"

# =====
# 2. LOAD SWIN BACKBONE & FREEZE
# =====
# This loads the base 1000-class model, swaps the head to 2-classes (hence the
↳ log warning)
swin_model = SwinForImageClassification.from_pretrained(
    "microsoft/swin-base-patch4-window7-224",
    num_labels=2,
    ignore_mismatched_sizes=True
).to(device)

# Now we overwrite those weights with YOUR custom fine-tuned 2-class weights
```

```

if os.path.exists(swin_checkpoint_path):
    checkpoint = torch.load(swin_checkpoint_path, map_location=device)
    # Check if your checkpoint saved raw weights or a utility dict
    if 'model_state_dict' in checkpoint:
        swin_model.load_state_dict(checkpoint['model_state_dict'])
    else:
        swin_model.load_state_dict(checkpoint)
    print("SUCCESS: Custom Swin Checkpoint loaded successfully!")
else:
    print(f"ERROR: Swin Checkpoint STILL not found at {swin_checkpoint_path}")

# Freeze Swin entirely
for param in swin_model.parameters():
    param.requires_grad = False
swin_model.eval()

# =====
# 3. LOAD FREQUENCY BRANCH & FREEZE
# =====
freq_branch = FrequencyBranch(embed_dim=768).to(device)

if os.path.exists(freq_branch_checkpoint_path):
    checkpoint = torch.load(freq_branch_checkpoint_path, map_location=device)
    if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
        freq_branch.load_state_dict(checkpoint['model_state_dict'])
    else:
        freq_branch.load_state_dict(checkpoint)
    print("SUCCESS: Frequency Branch loaded successfully!")
else:
    print(f"ERROR: Frequency Branch checkpoint STILL not found at_
↪{freq_branch_checkpoint_path}")

# Freeze Frequency Branch
for param in freq_branch.parameters():
    param.requires_grad = False
freq_branch.eval()

# =====
# 4. INITIALIZE TRAINABLE FUSION HEAD
# =====
# The Transformer expects the total combined dimension: 1024 + 768 = 1792
video_head = SwinTemporalTransformer(input_dim=1792).to(device)

# Optimizer ONLY targets the Fusion Head parameters

```

```
optimizer = torch.optim.AdamW(video_head.parameters(), lr=1e-6,
    ↪weight_decay=1e-4)
criterion = nn.CrossEntropyLoss()
```

Loading weights: 0%| | 0/449 [00:00<?, ?it/s]

SwinForImageClassification LOAD REPORT from: microsoft/swin-base-patch4-window7-224

Key	Status
classifier.weight	MISMATCH
classifier.bias	MISMATCH

```
-----+-----+-----
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt:
torch.Size([1000, 1024]) vs model:torch.Size([2, 1024])
classifier.bias   | MISMATCH | Reinit due to size mismatch - ckpt:
torch.Size([1000]) vs model:torch.Size([2])
```

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

SUCCESS: Custom Swin Checkpoint loaded successfully!

SUCCESS: Frequency Branch loaded successfully!

```
[14]: print("Class to Index Mapping:", train_dataset.class_to_idx)
```

Class to Index Mapping: {'Fake': 0, 'Real': 1}

```
[15]: import torch
import torch.optim as optim
from tqdm import tqdm

# 1. Freeze Spatial (Swin) and Frequency Backbones
swin_model.eval()
freq_branch.eval()

for param in swin_model.parameters():
    param.requires_grad = False
for param in freq_branch.parameters():
    param.requires_grad = False

# 2. Only Temporal Head is Trainable
video_head.train()
optimizer = optim.AdamW(video_head.parameters(), lr=2e-5, weight_decay = 1e-4)
    ↪# Warmup LR suggested [cite: 615]
criterion = torch.nn.CrossEntropyLoss()
```

```
[16]: import json
import torch
import torch.nn as nn
```

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score
from torch.utils.data import WeightedRandomSampler, DataLoader
from tqdm.auto import tqdm

# --- INITIAL SETUP ---
NUM_EPOCHS = 10
BATCH_SIZE = 8
JSON_PATH = "swin_temporal_history.json"
VIDEO_WEIGHTS_PATH = "swin_best_temporal.pth"
device = "cuda" if torch.cuda.is_available() else "cpu"
best_val_f1 = 0.0
history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

# =====
#   THE NUCLEAR FIX: 50/50 BATCH SAMPLER
# =====
print("Building the WeightedRandomSampler to force 50/50 batches...")

# 1. Grab all labels from your SwinVideoDataset
train_labels = [train_dataset.class_to_idx[label_str] for _, label_str in
    ↪train_dataset.video_folders]

# 2. Calculate inverse weights
count_fake = train_labels.count(0)
count_real = train_labels.count(1)
weight_fake = 1.0 / count_fake
weight_real = 1.0 / count_real

# 3. Assign weights to every video
sample_weights = [weight_fake if label == 0 else weight_real for label in
    ↪train_labels]

# 4. Create the Sampler
sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)

# 5. Re-initialize the DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    sampler=sampler, # Shuffle is automatically disabled by sampler
    num_workers=4,

```

```

        pin_memory=True
    )

    # Balanced loss (no heavy nudging needed because sampler handles the balance)
    criterion = nn.CrossEntropyLoss().to(device)

```

Building the WeightedRandomSampler to force 50/50 batches...

```

[18]: import torch
import numpy as np
from sklearn.metrics import f1_score, precision_score, recall_score
from tqdm.auto import tqdm

# 1. Load your saved checkpoint
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

video_head.load_state_dict(torch.load("checkpoints/swin_temporal_epoch_15.pth",
    ↪map_location=device))
video_head.eval()
swin_model.eval()
freq_branch.eval()

# 2. Collect raw probabilities on validation set
all_probs = [] # probability of being "Real" (class index 1)
all_labels = []

with torch.no_grad():
    for video_sequences, labels in tqdm(val_loader, desc="Collecting probs"):
        B, F, C, H, W = video_sequences.shape
        video_sequences = video_sequences.to(device)

        flat_frames = video_sequences.view(-1, C, H, W)

        # Feature extraction (same as training)
        spatial = swin_model.swin(pixel_values=flat_frames).pooler_output
        freq = freq_branch(flat_frames)
        fused = torch.cat([spatial, freq], dim=1)
        temporal_input = fused.view(B, F, -1)

        logits = video_head(temporal_input) # [B, 2]
        probs = torch.softmax(logits, dim=1)[: , 1] # P(Real)

        all_probs.extend(probs.cpu().numpy())
        all_labels.extend(labels.numpy())

all_probs = np.array(all_probs)
all_labels = np.array(all_labels)

```

```

# 3. Sweep thresholds
print(f"\n{'Threshold':>10} {'F1':>8} {'Precision':>10} {'Recall':>8}")
print("-" * 42)

best_f1, best_thresh = 0.0, 0.5

for thresh in np.arange(0.10, 0.91, 0.05):
    preds = (all_probs >= thresh).astype(int)
    f1     = f1_score(all_labels, preds, zero_division=0)
    prec   = precision_score(all_labels, preds, zero_division=0)
    rec    = recall_score(all_labels, preds, zero_division=0)

    marker = "  best" if f1 > best_f1 else ""
    print(f"{thresh:>10.2f} {f1:>8.4f} {prec:>10.4f} {rec:>8.4f}{marker}")

    if f1 > best_f1:
        best_f1, best_thresh = f1, thresh

print(f"\n Optimal threshold: {best_thresh:.2f} | Best F1: {best_f1:.4f}")

# 4. Use this threshold at inference time
# Replace your usual argmax with:
# probs = torch.softmax(logits, dim=1)[: , 1]
# preds = (probs >= best_thresh).long()

```

Collecting probs: 0% | 0/682 [00:00<?, ?it/s]

```

-----
IndexError                                Traceback (most recent call last)
Cell In[18], line 19
     16 all_labels = []
     18 with torch.no_grad():
--> 19
    ↪   for video_sequences, labels in tqdm(val_loader, desc=
     20         B, F, C, H, W = video_sequences.shape
     21         video_sequences = video_sequences.to(device)

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/tqdm/notebook.py:250,
    ↪ in tqdm_notebook.__iter__(self)
     248 try:
     249     it = super().__iter__()
--> 250     yield from it
     251 # NB: except ... [ as ...] breaks IPython async KeyboardInterrupt
     252 except: # NOQA

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/tqdm/std.py:1181, in
    ↪ tqdm.__iter__(self)

```

```

1178 time = self._time
1180 try:
-> 1181     for obj in iterable:
1182         yield obj
1183         # Update and possibly print the progressbar.
1184         # Note: does not call self.update(1) for speed optimisation.

```

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/utils/data/
↳ dataloader.py:741, in _BaseDataLoaderIter.__next__(self)

```

738 if self._sampler_iter is None:
739     # TODO(https://github.com/pytorch/pytorch/issues/76750)
740     self._reset() # type: ignore[call-arg]
--> 741 data = self._next_data()
742 self._num_yielded += 1
743 if (
744     self._dataset_kind == _DatasetKind.Iterable
745     and self._IterableDataset_len_called is not None
746     and self._num_yielded > self._IterableDataset_len_called
747 ):

```

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/utils/data/
↳ dataloader.py:1548, in _MultiProcessingDataLoaderIter._next_data(self)

```

1546 worker_id = self._task_info.pop(idx)[0]
1547 self._rcvd_idx += 1
-> 1548 return self._process_data(data, worker_id)

```

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/utils/data/
↳ dataloader.py:1586, in _MultiProcessingDataLoaderIter._process_data(self,
↳ data, worker_idx)

```

1584 self._try_put_index()
1585 if isinstance(data, ExceptionWrapper):
-> 1586     data.reraise()
1587 return data

```

File ~/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/_utils.py:775,
↳ in ExceptionWrapper.reraise(self)

```

771 except Exception:
772     # If the exception takes multiple arguments or otherwise can't
773     # be constructed, don't try to instantiate since we don't know how
774     raise RuntimeError(msg) from None
--> 775 raise exception

```

IndexError: Caught IndexError in DataLoader worker process 0.

Original Traceback (most recent call last):

```

File "/home/sai/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/util /
↳ data/_utils/worker.py", line 358, in _worker_loop
    data = fetcher.fetch(index) # type: ignore[possibly-undefined]
    ~~~~~

```

```

File "/home/sai/anaconda3/envs/tf_gpu/lib/python3.12/site-packages/torch/util/
↳data/_utils/fetch.py", line 54, in fetch
    data = [self.dataset[idx] for idx in possibly_batched_index]
           ~~~~~~^~~~~~
File "/tmp/ipykernel_556/4275315485.py", line 33, in __getitem__
    frames_paths += [frames_paths[-1]] * (self.num_frames - len(frames_paths))
                   ~~~~~~^~~~~~
IndexError: list index out of range

```

```

[ ]: =====
#                               TRAINING LOOP
# =====
for epoch in range(NUM_EPOCHS):
    print(f"\n--- Epoch {epoch+1}/{NUM_EPOCHS} ---")

    # --- TRAINING PHASE ---
    # We only train the video_head; Swin and Freq must stay frozen
    video_head.train()
    swin_model.eval()
    freq_branch.eval()

    train_loss, train_preds, train_labels_list = 0, [], []

    # Progress Bar (Jupyter style)
    train_bar = tqdm(train_loader, desc=f"Epoch {epoch+1} Training",
↳dynamic_ncols=True)

    for video_sequences, labels in train_bar:
        B, F, C, H, W = video_sequences.shape
        video_sequences, labels = video_sequences.to(device), labels.to(device)

        # Flatten frames for backbones: [B*16, 3, 224, 224]
        flat_frames = video_sequences.view(-1, C, H, W)

        optimizer.zero_grad()

        # FEATURE EXTRACTION (Frozen)
        with torch.no_grad():
            # 1. Swin Spatial (1024-D)
            spatial = swin_model.swin(pixel_values=flat_frames).pooler_output
            # 2. Frequency (768-D)
            freq = freq_branch(flat_frames)
            # 3. Concat and reshape to [Batch, 16, 1792]
            fused = torch.cat([spatial, freq], dim=1)
            temporal_input = fused.view(B, F, -1)

```

```

# TEMPORAL TRAINING
logits = video_head(temporal_input)
loss = criterion(logits, labels)

loss.backward()
optimizer.step()

# Metrics
train_loss += loss.item()
train_preds.extend(torch.argmax(logits, dim=1).cpu().tolist())
train_labels_list.extend(labels.cpu().tolist())
train_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_train_loss = train_loss / len(train_loader)
train_acc = accuracy_score(train_labels_list, train_preds)
train_f1 = f1_score(train_labels_list, train_preds, average='macro')

# --- VALIDATION PHASE ---
video_head.eval()
val_loss, val_preds, val_labels_list = 0, [], []

val_bar = tqdm(val_loader, desc="Validating", dynamic_ncols=True)
with torch.no_grad():
    for video_sequences, labels in val_bar:
        # --- MEMORY-SAFE FEATURE EXTRACTION ---
        # 1. Inside your Training / Validation Loop:
        # B=Batch, F=Frames, C=Channels, H=Height, W=Width
        video_sequences = video_sequences.to(device)
        labels = labels.to(device) # <--- ADD OR CHECK THIS LINE
        B, F, C, H, W = video_sequences.shape
        flat_frames = video_sequences.view(-1, C, H, W) # [128, 3, 224, 224]
        ↪224] if Batch=8

        spatial_list = []
        freq_list = []

        # Process frames in small "Chunks" to prevent OOM
        CHUNK_SIZE = 8 # Adjust this to 4 if it still crashes
        with torch.no_grad():
            for i in range(0, flat_frames.size(0), CHUNK_SIZE):
                chunk = flat_frames[i : i + CHUNK_SIZE].to(device)

                # Extract Spatial (Swin)
                swin_out = swin_model.swin(pixel_values=chunk).pooler_output
                spatial_list.append(swin_out)

```

```

        # Extract Frequency
        freq_out = freq_branch(chunk)
        freq_list.append(freq_out)
        del chunk

    # Reassemble the full sequence for the Temporal Transformer
    all_spatial = torch.cat(spatial_list, dim=0) # [128, 1024]
    all_freq = torch.cat(freq_list, dim=0)       # [128, 768]

    # Concatenate branches and reshape to [Batch, Frames,
↪Total_Features]
    fused = torch.cat([all_spatial, all_freq], dim=1) # [128, 1792]
    temporal_input = fused.view(B, F, -1)             # [8, 16,
↪1792]

    # 2. Now run your trainable video_head
    logits = video_head(temporal_input)
    loss = criterion(logits, labels)

    val_loss += loss.item()
    val_preds.extend(torch.argmax(logits, dim=1).cpu().tolist())
    val_labels_list.extend(labels.cpu().tolist())

    epoch_val_loss = val_loss / len(val_loader)
    val_acc = accuracy_score(val_labels_list, val_preds)
    val_f1 = f1_score(val_labels_list, val_preds, average='macro')

    print(f"Train Loss: {epoch_train_loss:.4f} | Train Acc: {train_acc:.4f} |
↪Train F1: {train_f1:.4f}")
    print(f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {val_acc:.4f} | Val F1:
↪{val_f1:.4f}")

    # --- LOGGING & SAVE BEST ---
    history["train_loss"].append(epoch_train_loss)
    history["val_loss"].append(epoch_val_loss)
    history["train_acc"].append(train_acc)
    history["val_acc"].append(val_acc)

    with open(JSON_PATH, "w") as f:
        json.dump(history, f, indent=4)

    # Save checkpoint after each epoch
    epoch_weights_path = f"swin_temporal_epoch_{epoch+1}.pth"
    torch.save(video_head.state_dict(), epoch_weights_path)
    print(f"Checkpoint saved for epoch {epoch+1} at {epoch_weights_path}")

    if val_f1 > best_val_f1:

```

```

        best_val_f1 = val_f1
        torch.save(video_head.state_dict(), VIDEO_WEIGHTS_PATH)
        print(f" Best Model saved! (Macro F1: {best_val_f1:.4f})")

    # --- VISUALIZATION (Confusion Matrix) ---
    cm = confusion_matrix(val_labels_list, val_preds)
    plt.figure(figsize=(4, 3))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Fake", "Real"], yticklabels=["Fake", "Real"])
    plt.title(f"Val Confusion Matrix - Epoch {epoch+1}")
    plt.show()

print("\nTraining Complete!")

```

```

[1]: import os
import json
import torch
from safetensors.torch import save_file
from huggingface_hub.serialization import split_torch_state_dict_into_shards

# =====
# 1. SETUP PATHS AND CONFIGURATION
# =====
PTH_WEIGHTS_PATH = "checkpoints/swin_temporal_epoch_15.pth" # Path to your existing checkpoint
SAVE_DIRECTORY = "./temporal_sharded_safetensors_model" # Target directory for output
MAX_SHARD_SIZE = "100MB" # Adjustable chunk size

os.makedirs(SAVE_DIRECTORY, exist_ok=True)

# =====
# 2. LOAD AND CLEAN STATE DICTIONARY
# =====
print(f"Loading weights from {PTH_WEIGHTS_PATH}...")
raw_checkpoint = torch.load(PTH_WEIGHTS_PATH, map_location="cpu")

# Handle cases where weights are wrapped inside a training checkpoint dictionary
if isinstance(raw_checkpoint, dict) and "model_state_dict" in raw_checkpoint:
    print("Detected standard checkpoint wrapper. Extracting 'model_state_dict'...")
    state_dict = raw_checkpoint["model_state_dict"]
elif isinstance(raw_checkpoint, dict):
    state_dict = raw_checkpoint
else:
    raise TypeError("Loaded object is not a valid state dictionary layout.")

```

```

# Clean state_dict: Convert integers/scalars into PyTorch tensors to prevent .
↳ device errors
clean_state_dict = {}
for key, value in state_dict.items():
    if torch.is_tensor(value):
        clean_state_dict[key] = value
    elif isinstance(value, (int, float)):
        # Safe wrapper for BatchNorm trackers or metadata configurations
        clean_state_dict[key] = torch.tensor(value)
    else:
        # Skip string configurations or non-numeric metadata that safetensors_
↳ can't store
        print(f"Skipping non-serializable key: {key}")

# =====
# 3. COMPUTE SHARDING BLUEPRINT
# =====
print("Calculating sharding layout with huggingface_hub...")

# By omitting filename_pattern, it safely defaults to the standard_
↳ 'model{suffix}.safetensors'
split_info = split_torch_state_dict_into_shards(
    clean_state_dict,
    max_shard_size=MAX_SHARD_SIZE
)

# =====
# 4. EXECUTE SHARD SAVING & INDEXING
# =====
print(f"Writing shards to: {SAVE_DIRECTORY}")

# Loop through the blueprint mapping files to lists of tensor keys
for filename, tensor_names in split_info.filename_to_tensors.items():
    shard_path = os.path.join(SAVE_DIRECTORY, filename)

    # Isolate only the tensors assigned to this specific shard
    shard_state_dict = {name: clean_state_dict[name] for name in tensor_names}

    # Save natively as safetensors
    save_file(shard_state_dict, shard_path, metadata={"format": "pt"})
    print(f" Saved: {filename} containing {len(tensor_names)} tensors.")

# Generate and save the metadata index mapping file
index_data = {
    "metadata": {"total_size": sum(t.nelement() * t.element_size() for t in_
↳ clean_state_dict.values())},
    "weight_map": split_info.tensor_to_filename

```

```

}

index_path = os.path.join(SAVE_DIRECTORY, "model.safetensors.index.json")
with open(index_path, "w", encoding="utf-8") as f:
    json.dump(index_data, f, indent=2)

print(f" Successfully created sharded weights layout and 'model.safetensors.
↪index.json' map!")

```

```

Loading weights from checkpoints/swin_temporal_epoch_15.pth...
Calculating sharding layout with huggingface_hub...
Writing shards to: ./temporal_sharded_safetensors_model
Saved: model-00001-of-00004.safetensors containing 13 tensors.
Saved: model-00002-of-00004.safetensors containing 12 tensors.
Saved: model-00003-of-00004.safetensors containing 12 tensors.
Saved: model-00004-of-00004.safetensors containing 18 tensors.
Successfully created sharded weights layout and 'model.safetensors.index.json'
map!

```